

EXPERIMENT - 7

Aim

To understand the fundamental structure and syntax of Solidity smart contracts, explore core programming constructs such as data types, functions, modifiers, and control structures, and gain practical exposure to writing, compiling, deploying, and testing smart contracts using the Remix IDE development environment.

Theory

Solidity is a statically-typed, contract-oriented programming language designed specifically for implementing smart contracts on Ethereum and other EVM-compatible blockchains. A smart contract is essentially self-executing code deployed on the blockchain, containing rules and logic that run automatically once triggered by a transaction.

Every Solidity file typically begins with an SPDX license identifier and a pragma directive specifying the compiler version, followed by the contract body containing state variables, a constructor, optional modifiers, and functions.

Data Types and Variables

Solidity supports basic types like `uint/int` (integers), `bool` (true/false), `address` (blockchain account references), `string` (text), and `bytes` (raw binary data). Variables fall into three categories: state variables (permanently stored on-chain), local variables (temporary, function-scoped), and global variables (blockchain context like `msg.sender`, `msg.value`, `block.timestamp`).

Functions

Functions hold a contract's executable logic. `pure` functions neither read nor write state; `view` functions may read state but not modify it. Functions can take parameters as input and return one or more computed values.

Visibility

Four visibility levels control access: `public` (open to all callers), `private` (contract-only), `internal` (contract and its derived contracts), and `external` (callable only from outside).

Modifiers and Constructors

Modifiers wrap reusable pre-conditions around functions — commonly used for access control (e.g., restricting a function to the contract owner). A constructor runs once, at deployment, typically to initialize state.

Control Flow and Data Structures

Standard constructs like `if-else`, `for`, `while`, and `do-while` control program flow, though loops must be used cautiously since every operation consumes gas. Solidity also offers arrays (ordered lists), mappings (key-value stores), structs (grouped custom types), and enums (fixed named constants).

Data Locations

Data can reside in `storage` (persistent, on-chain), `memory` (temporary, function-scoped), or `calldata` (read-only, used for external function inputs).

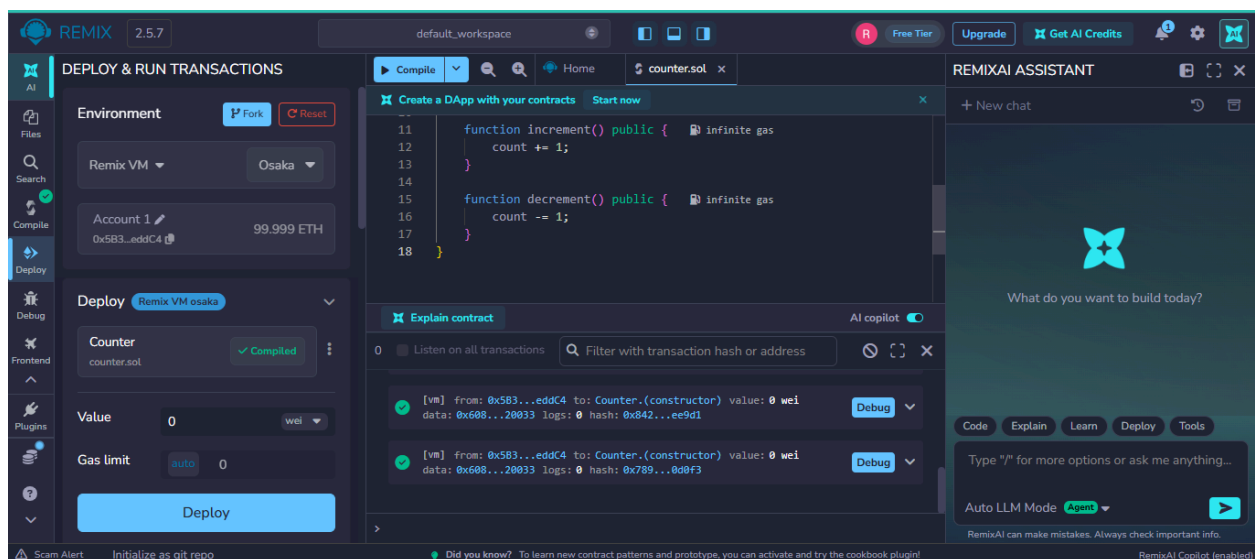
Ether, Gas, and Transactions

Ether is Ethereum's native currency, with Wei as its smallest denomination ($1 \text{ ETH} = 10^{18} \text{ Wei}$). Every operation on-chain costs gas, and transaction fees are computed as gas used multiplied by gas price — incentivizing efficient contract design. Functions that accept Ether must be marked `payable`.

Remix IDE

Remix is a browser-based IDE that supports the full smart-contract lifecycle: writing code, compiling, deploying (via a simulated Remix VM or a live network), and interacting with deployed contracts through its UI.

OUTPUT:



REMIX 2.5.7 default_workspace

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, Account 1 (0x5B3...eddC4), 99.999 ETH

Functions: Select a function to interact with... Low level interaction: +

• get

0xuint256: 0

3 New Deployments

counter.sol

```
11 function increment() public { infinite gas
12   count += 1;
13 }
14
15 function decrement() public { infinite gas
16   count -= 1;
17 }
18 }
```

REMIXAI ASSISTANT

+ New chat

What do you want to build today?

Code Explain Learn Deploy Tools

Type "/" for more options or ask me anything...

Auto LLM Mode Agent

REMIX 2.5.7 default_workspace

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, Account 1 (0x5B3...eddC4), 99.999 ETH

Low level interaction: +

• increment

Value: 0 wei

Gas limit: auto 0

Transact

Balance: 0 ETH

counter.sol

```
11 function increment() public { infinite gas
12   count += 1;
13 }
14
15 function decrement() public { infinite gas
16   count -= 1;
17 }
18 }
```

REMIXAI ASSISTANT

+ New chat

What do you want to build today?

Code Explain Learn Deploy Tools

Type "/" for more options or ask me anything...

Auto LLM Mode Agent

REMIX 2.5.7 default_workspace

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, Account 1 (0x5B3...eddC4), 99.999 ETH

Low level interaction: +

• get

0xuint256: 1

Call

Balance: 0 ETH

counter.sol

```
11 function increment() public { infinite gas
12   count += 1;
13 }
14
15 function decrement() public { infinite gas
16   count -= 1;
17 }
18 }
```

REMIXAI ASSISTANT

+ New chat

What do you want to build today?

Code Explain Learn Deploy Tools

Type "/" for more options or ask me anything...

Auto LLM Mode Agent

REMIX 2.5.7 default_workspace

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, Account 1 (0x583...eddC4), 99.999 ETH

Low level interaction: decrement

Value: 0, Gas limit: auto, 0

Transact

Balance: 0 ETH

counter.sol

```
11 function increment() public {
12     count += 1;
13 }
14
15 function decrement() public {
16     count -= 1;
17 }
18 }
```

Explain contract

AI copilot

revert

The transaction has been reverted to the initial state. Note: The called function should be payable if you send value and the value you send should be 1. If the transaction failed for not having enough gas, try increasing the gas limit gently. If the transaction failed for not having enough gas, try increasing the gas limit gently.

REMIXAI ASSISTANT

+ New chat

What do you want to build today?

Code Explain Learn Deploy Tools

Type "/" for more options or ask me anything...

Auto LLM Mode Agent

RemixAI can make mistakes. Always check important info.

RemixAI Copilot (enabled)

Welcome to the new home of Remix. Bookmark app.remix.live so you always land here. [Still need your files from remix.ethereum.org?](https://app.remix.live)

REMIX 2.5.7 default_workspace

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, Account 1 (0x583...eddC4), 99.999 ETH

Select a function to interact with...

Low level interaction

get

0xint256: 0

Call

Balance: 0 ETH

counter.sol

```
11 function increment() public {
12     count += 1;
13 }
14
15 function decrement() public {
16     count -= 1;
17 }
18 }
```

Explain contract

AI copilot

Listen on all transactions

Filter with transaction hash or address

CALL [call] from: 0x58380a6a701c568545dcfc803fc8875f56beddc4 to: Counter.get() data: 0x6d4...ce63c

Debug

REMIXAI ASSISTANT

+ New chat

What do you want to build today?

Code Explain Learn Deploy Tools

Type "/" for more options or ask me anything...

Auto LLM Mode Agent

RemixAI can make mistakes. Always check important info.

RemixAI Copilot (enabled)

DEPLOY & RUN TRANSACTIONS

Environment: Solidity compiler, Osaka, sanket (0x583...eddC4), 99.999 ETH

SimpleStorage

Functions

Select a function to interact with...

Low level interaction

num

0xint256: 50

variables.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // PUSH1 costs 3 gas - this line costs 96 gas - 2978936 gas left
6     // State variable to store a number
7     uint public num;
8
9     // You need to send a transaction to write to a state variable.
10    function set(uint _num) public {
11        num = _num;
12    }
13
14    // You can read from a state variable without sending a transaction.
15    function get() public view returns (uint) {
16        return num;
17    }
18 }
```

Explain contract

AI copilot

Listen on all transactions

Filter with transaction hash or address

CALL [call] from: 0x58380a6a701c568545dcfc803fc8875f56beddc4 to: SimpleStorage.num() data: 0x4e7...0b1dc

Debug

CALL [call] from: 0x58380a6a701c568545dcfc803fc8875f56beddc4 to: SimpleStorage.num() data: 0x4e7...0b1dc

Debug

REMIXAI ASSISTANT

+ New chat

What do you want to build today?

Code Explain Learn Deploy Tools

Type "/" for more options or ask me anything...

Auto LLM Mode Agent

RemixAI can make mistakes. Always check important info.

RemixAI Copilot (enabled)

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, Account 1 (0x5B3...eddC4), 99.999 ETH

Functions: Select a function to interact with...

Low Level interaction: +

• x

0:uint256: 1

Call

Compiled: tax_test.sol, variables.sol, readAndWrite.sol, viewAndPure.sol, modifiersAndConstructors.sol

Create a DApp with your contracts Start now

```
9   return x + y;
10  }
11
12  // Promise not to modify or read from the state.
13  function add(uint i, uint j) public pure returns (uint) { infinite gas
14      return i + j;
15  }
16 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Fetchd compilation data for 0xd9145CC52D386F254917e481e844e9943F39138 from cache (CompilerArtefacts)

[vm] from: 0x5B3...eddC4 to: ViewAndPure.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x774...ec915 Debug

[call] from: 0x5B380a6a701c56854dcFcB83FcB875F56beddC4 to: ViewAndPure.x() data: 0x0c5...5699c Debug

Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'.

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, Account 1 (0x5B3...eddC4), 99.999 ETH

Functions: Select a function to interact with...

Low Level interaction: +

• addToX uint256

5

Calldata Parameters

0:uint256: 6

Call

Balance 0 ETH

Compiled: tax_test.sol, variables.sol, readAndWrite.sol, viewAndPure.sol, modifiersAndConstructors.sol

Create a DApp with your contracts Start now9 return x + y;
10 }
11
12 // Promise not to modify or read from the state.
13 function add(uint i, uint j) public pure returns (uint) { infinite gas
14 return i + j;
15 }
16 }

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

[call] from: 0x5B380a6a701c56854dcFcB83FcB875F56beddC4 to: ViewAndPure.x() data: 0x0c5...5699c Debug

[call] from: 0x5B380a6a701c56854dcFcB83FcB875F56beddC4 to: ViewAndPure.addToX(uint256) data: 0x596...00005 Debug

Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'.

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, Account 1 (0x5B3...eddC4), 99.999 ETH

Functions: Select a function to interact with...

Low Level interaction: +

• x

0:uint256: 10

Call

Balance 0 ETH

Compiled: variables.sol, readAndWrite.sol, viewAndPure.sol, modifiersAndConstructors.sol, inputsAndOutputs.sol

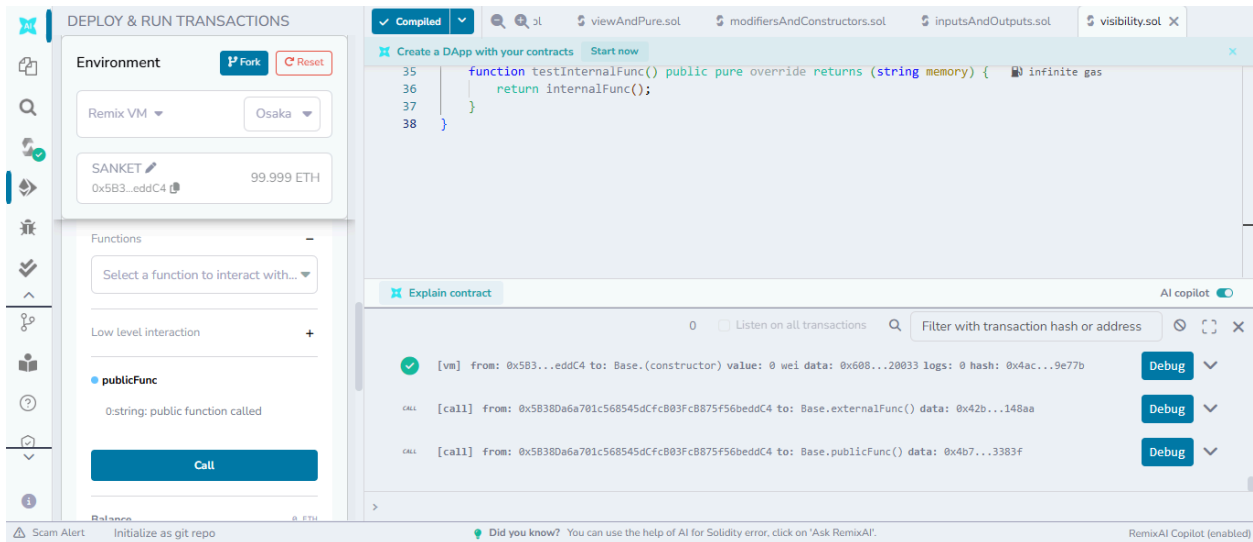
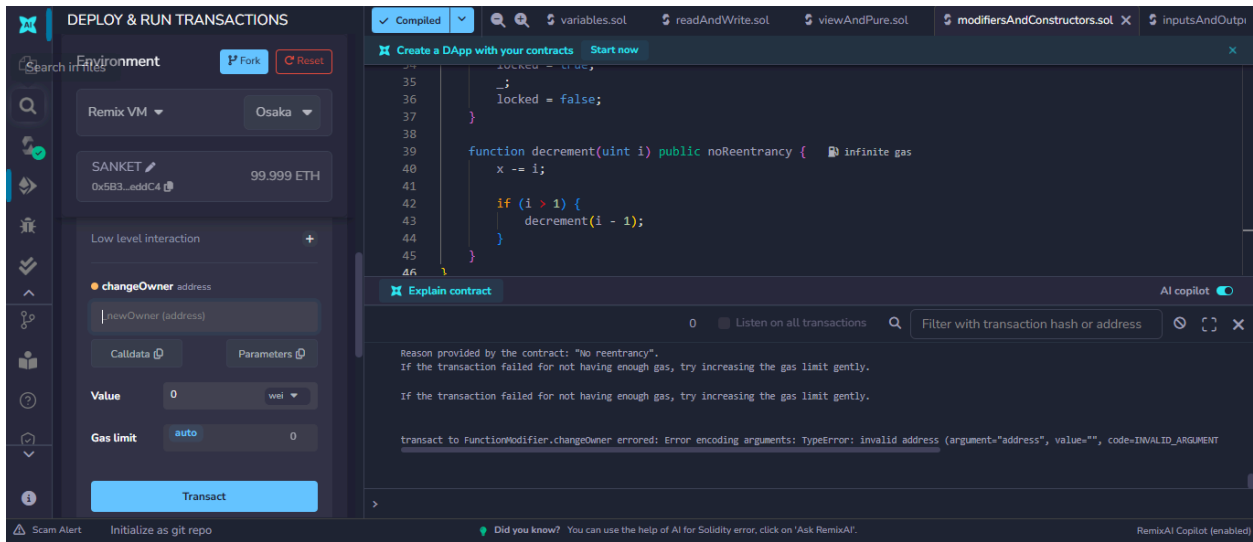
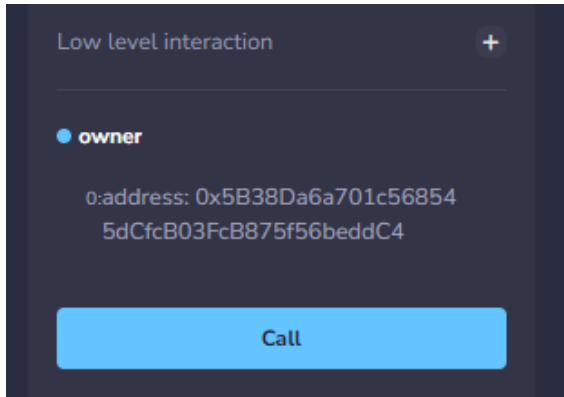
Create a DApp with your contracts Start now34 locked = true;
35 _;
36 locked = false;
37 }
38
39 function decrement(uint i) public noReentrancy { infinite gas
40 x -= i;
41
42 if (i > 1) {
43 decrement(i - 1);
44 }
45 }
46 }

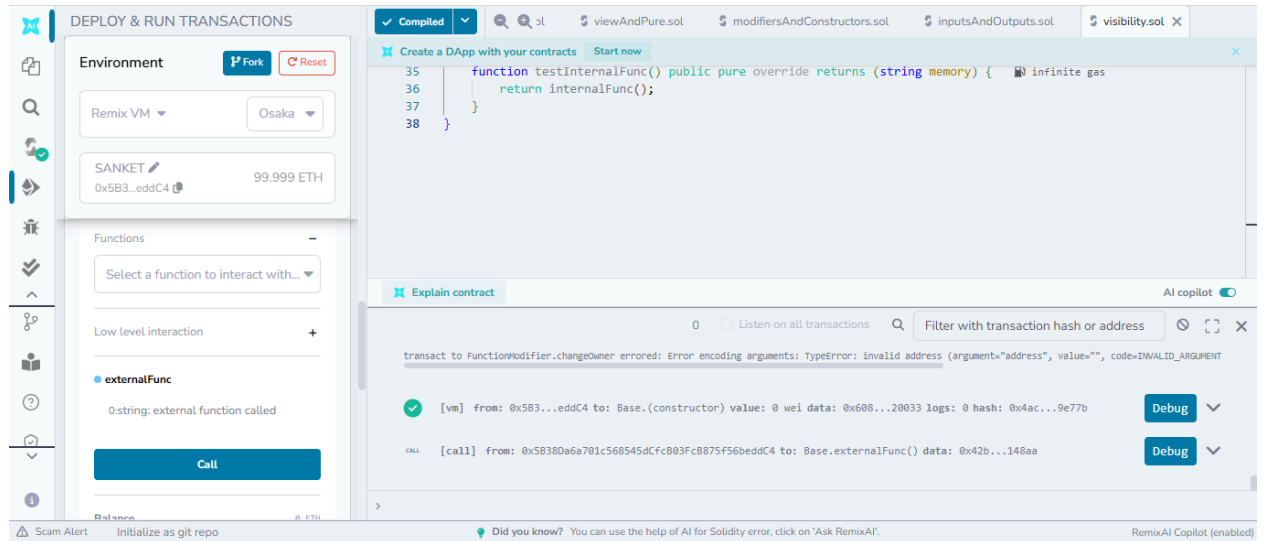
Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'.





Conclusion

This experiment provided a solid, hands-on introduction to Solidity and the Remix development workflow. By working through the Solidity Basics Course's 19 guided assignments, key language concepts — data types, variable scopes, function types, visibility rules, modifiers, constructors, control-flow statements, and data structures like arrays, mappings, structs, and enums — were reinforced through direct coding practice rather than theory alone.

Deploying contracts on the Remix VM and calling their functions (both read-only and state-changing) clarified how transactions actually alter blockchain state and consume gas. Overall, the exercise built practical confidence in writing, compiling, deploying, and debugging Solidity smart contracts, laying the groundwork for more advanced blockchain development.